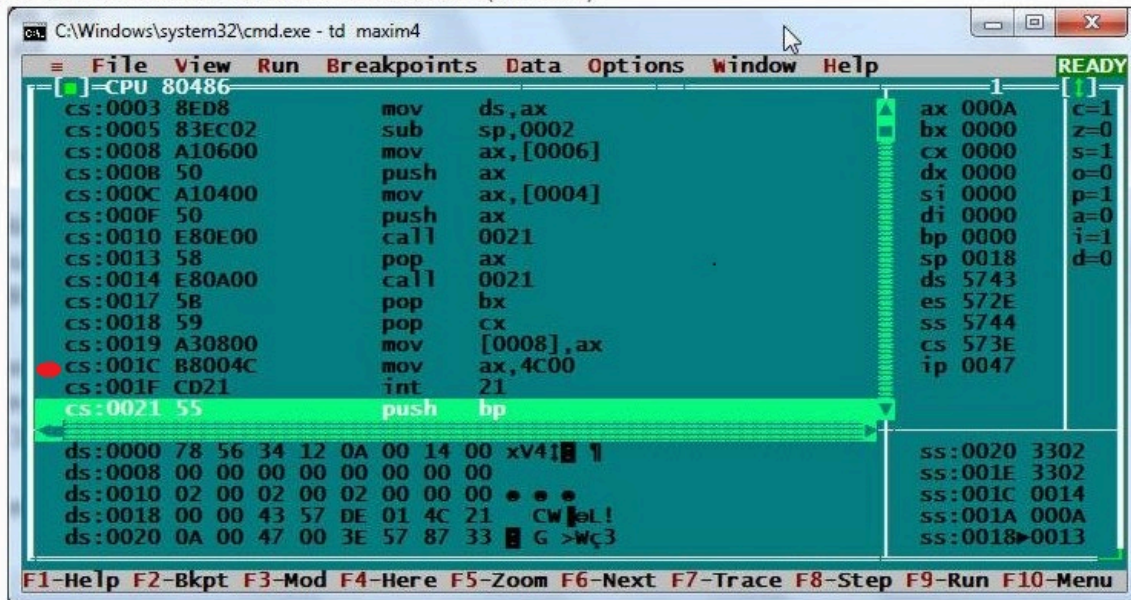
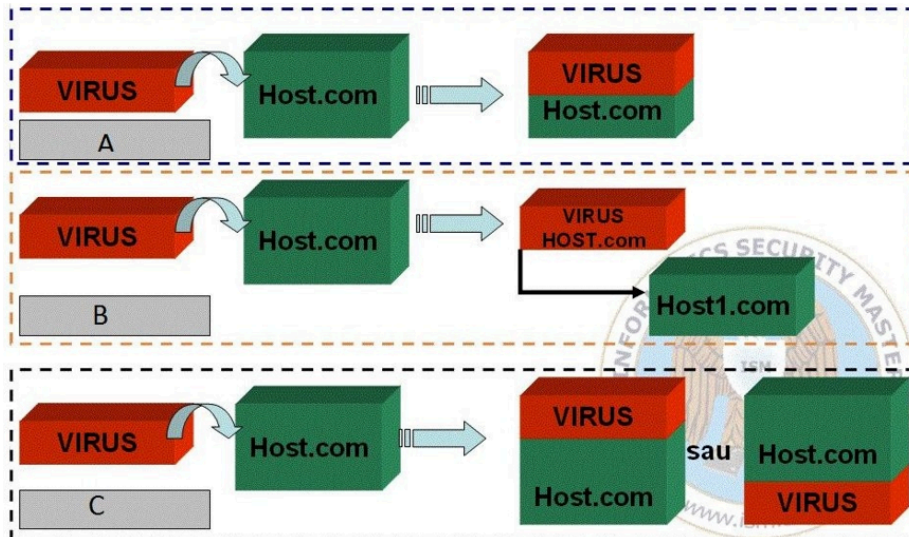


1. Considering the next image that describes the process state, and the current executed statement ret 5, indicate which is the next instruction to be executed (after ret 5):



- A. sub SP, 0002
- B. pop CX
- C. none of these answers
- D. pop BX
- E. pop AX

2. Indicate the correct type of the viruses described in next image:



- A. A - parasitic, B - overwriting, C - companion
- B. A - overwriting, B - parasitic, C - companion
- C. A - companion, B - parasitic, C - overwriting
- D. A - overwriting, B - companion, C - parasitic
- E. A - parasitic, B - companion, C - overwriting

3. It is given the macrodefinition (left) and its call (right) in the next image:
What is the result obtained in AX and BX after its execution:

<pre> Swap01 MACRO X, Y push AX push BX mov BX, X mov AX, Y mov X, AX mov Y, BX pop BX pop AX ENDM </pre>	<pre> vb2 DW 1234h ... mov AX, 1111h mov BX, 2222h swap01 AX, vb2 ... </pre>
---	--

- A. AX = 1234h and BX = 2222h
- B. AX = 2222 and BX = 1111h
- C. The result can NOT be determined because the source code has errors
- D. AX = 1111h and BX = 2222h
- E. AX = 2222h and BX = 1234h

4. What is a stack-frame?

- A. None of these answers
- B. A run-time exception generated by a small stack
- C. A memory zone created in the data segment
- D. A memory zone created in the stack segment for local variables
- E. A special area in the .com process header, like the DTA
- F. The stack image at a precise moment during the process execution

5. 2 DB type arrays size equal to 10 (they have their addresses stored in DS:SI and ES:DI) are copied from source into destination. Which instruction block is going to do the copy:

<pre> 1. mov CX, 10 ciclu: mov AL, [SI] mov [DI], AL INC SI INC DI loop ciclu </pre>	<pre> 2. mov CX, 10 ciclu: movsb loop ciclu </pre>	<pre> 3. mov CX, 10 rep movsb </pre>
--	--	--------------------------------------

- A. only 2
- B. only 2 and 3
- C. only 1 and 2
- D. only 3
- E. only 1
- F. all

6. File viruses can infect:

- a. BOOT**
- b. driver and/or executable**
- c. C++ source file**
- d. Java source file**

- A. a + b + c + d**
- B. b + c**
- C. a + d**
- D. a + b**
- E. b + c + d**

7. Knowing that the execution of a software interrupt, the microprocessor:

- b. Seek the interrupt address in the interrupt vector table**
- a. Sets 0 in the TF (trap flag) and IF (interrupt enable file) flag bits**
- d. Saves CS and IP of the calling program on the stack (the return address)**
- c. Does a FAR jump in the interrupt handler**
- f. Run the routine code until it encounters IRET**
- e. Restore IP, CS and set flag bits**

Select the answer that indicate the correct logical order of the above sequence:

- A. a + b + d + c + f + e**
- B. a + b + c + d + e + f**
- C. a + b + d + e + c**
- D. None of these answers**
- E. b + a + d + c + f + e**
- F. b + a + d + e + c**

8. Which one from the next statements it is NOT an indexed addressing mode:

- A. MOV AX, SI**
- B. MOV AX, [SI][DI]**
- C. MOV AX, [SI]**
- D. MOV AX, [SI + 2]**
- E. MOV AX, [SI][2]**

9. During a FAR procedure call, the procedure arguments and the returning address are put on the stack from right to left (for arguments). For an Intel 8086 values are stored in memory in little-endian/big-endian format.

Considering the next code sequence:

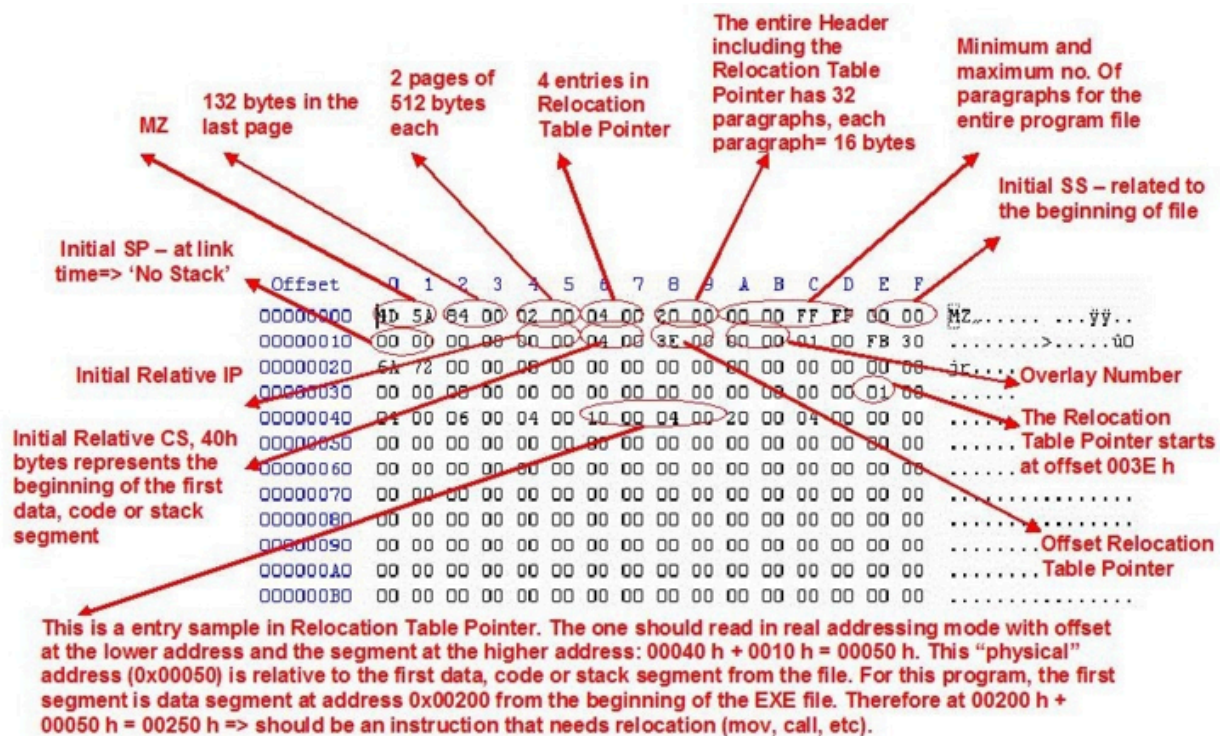
```
.....  
mov SP, 000Eh  
mov AX, 7755h  
push AX  
mov AX, 1234h  
push AX  
mov AX, 1133h  
push AX  
call far ptr SALT
```

```
.....  
SALT:  
push BP  
mov BP, SP  
push AX  
mov AX, [BP + 8]  
.....
```

indicate what value is stored in AX by the last instruction:

- A. none of these answers
- B. the value of the returning IP
- C. the value can't be determined because it is outside the stack (using BP indexed addressing mode you access data only from DS - data segment)
- D. 1133h
- E. 7755h
- F. 1234h

10. Knowing the structure of headers for 16-bit DOS executable (the next image). Specify for how many instructions in the source code, the OS need to readjust their internal code (because an EXE file "doesn't knows" what values will have the segment registers DS, ES, CS, SS):



- A. 4 instructions
- B. 2 instructions
- C. 1 instruction
- D. none of these answers
- E. 16 instructions

11. To illustrate the procedure for "reverse engineering" in ASM 8086, you receive a ASM8086 code sequence that adds three integers (DW type) and stores the result in another variable (x = y + z + v).

You know that

- 0008h is the x variable offset
 - 0002h is the y variable offset
 - 0004h is the z variable offset
 - 0006h is the v variable offset.
 - the compiled form of assembler instructions is:
- ```

mov ax, 4C00h = B8 00 4C
mov ax, [0002] = A1 02 00 - muta y in AX
add ax, [0006] = 03 06 06 00 - adauga v la AX
add ax, [0004] = 03 06 04 00 - adauga z la AX
xor ax, ax = 33 C0
mov [0008], ax = A3 08 00 - stochează rezultatul în x
push ax = 50
int 21h = CD 21

```

Using the above information indicate which statement is missing from the next sequence that is implementing the addition:

```

.....
cs:0005 33 C0 - xor ax, ax
cs:0007 A1 02 00 - mov ax, [0002] (muta y in AX)
cs:000A 03 06 04 00 - add ax, [0004] (adauga z la AX)
cs:000E 03 06 06 00 - add ax, [0006] (adauga v la AX)
cs:0012 _____ ; <= complete with hex values representing the binary image
 of the instruction
cs:0015 B8 00 4C - mov ax, 4c00h (val pentru intoarcere)
cs:0018 CD 21 - int 21h

```

- .....
- A. mov [0008], ax = A3 08 00
  - B. add ax, [0004] = 03 06 04 00
  - C. add ax, [0006] = 03 06 06 00
  - D. mov ax, [0002] = A1 02 00
  - E. none of these instructions

12. Companion viruses can have the type:

- A. C source code file
- B. assembler source code file
- C. none of these types
- D. C++ source code file
- E. executable
- F. boot

**13. Considering the next code sequence:**

```
mov CX, 5
XOR AX, AX
```

**repeta:**

```
INC AX
JMP final
loop repeta
```

**final:**

```
mov DS:[0000], AX
```

**indicate the value that is stored at [0000] after the last instruction:**

- A. 5**
- B. 4**
- C. you can't determine it because the loop is infinite**
- D. 0**
- E. 1**

**14. For the next code sequence:**

```
mov CX, 5
XOR AX, AX
```

**repeat:**

```
INC CX
INC AX
loop repeat
```

**final:**

```
mov DS:[0000], AX
```

**Which is the value stored at [0000] offset after the sequence has been executed:**

- A.1**
- B.5**
- C.0**
- D. It is not possible to determine it because the sequence contains an infinite loop**
- E.4**

**15. It is NOT a feature of a memory resident viruses:**

- A. The virus hides in the memory & hooks a function of the interrupt 21 h**
- B. none of these answers**
- C. Inserts itself in the end of the .COM file**
- D. The virus can be a parasitic one**
- E. DOESN'T destroy the host program**

16. For the next assembler source code:

```
.stack 16
.data
 a DW 1111h
.code
 mov AX, @data
 mov DS, AX

 mov CX, 20
 mov AX, a
repeat:
 PUSH AX
 loop repeat
final:
 mov AX, 4c00h
 int 21h
end
```

select the correct statement:

- A. The source code is correct but the process will generate a Stack Overflow exception
- B. The source code has errors because the final label it is not used
- C. The source code has errors because the CX register it is not initialized correctly
- D. The source code is correct and the process will run without problems
- E. The source code has errors because the hex value, 1111 h, it's too large for a Word variable

17. For these 2 logic addresses:

address1 - 5732:001A

address2 - 572D:006A

indicate their physical order in memory, if you consider a Intel 8086 processor that uses a real addressing mode

- A. Address1 < address2
- B. the real addressing mode is not for Intel 8086 processors and you can't determine their physical addresses
- C. none of these answers
- D. Address1 > address2
- E. address1 and address2 represent identical physical addresses



18. Taking into consideration the next assembler sequence:

**.data**

vector DW 10, 20, 30, 40, 50

n DB 4

suma DW ?

**.code**

.....

mov AX, 0

push AX

mov AX, 10

push AX

mov AX, 11

push AX

.....

, the stack operations sequence is equivalent with calling a C function that has the header:

- A. void suma(int\* suma, int n, int 'vector)
- B. void suma(int\* suma, int \*n, int \*vector)
- C. void suma(int\* vector, int n, int \*suma)
- D. void suma(int\* vector, int \*n, int 'suma)
- E. void suma(int\* suma, int \*vector, int n)

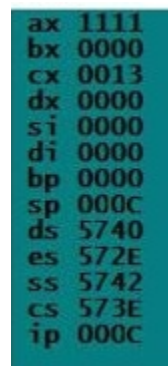
19. If you take into consideration the process state from the given image, indicate which will be the SP register value after the next instructions are executed:

mov AX, 1111h

push AX

push AX

- A. 0008h
- B. 000Ah
- C. 000Eh
- D. 0010h
- E. 1111h



|    |      |
|----|------|
| ax | 1111 |
| bx | 0000 |
| cx | 0013 |
| dx | 0000 |
| si | 0000 |
| di | 0000 |
| bp | 0000 |
| sp | 000C |
| ds | 5740 |
| es | 572E |
| ss | 5742 |
| cs | 573E |
| ip | 000C |

20. Considering the data area defined like this:

A1 DB 0

A2 DD 0

A3 DQ 0

A4 DW 0

A5 DT 0

and the next code sequence

xor SI, SI

mov AX, 1234h

add SI, 12

mov [SI], AX

mov SI, 8

mov [SI], AX

mov SI, 14

mov [SI], AX

indicate the final value of A4:

A. 1111h

B. 3412h

C. the value can not be determined because there are errors

D. 0000h

E. 1234h

21. Considering the next data declarations:

. data

a DB ?

b DD ?

c DW ?

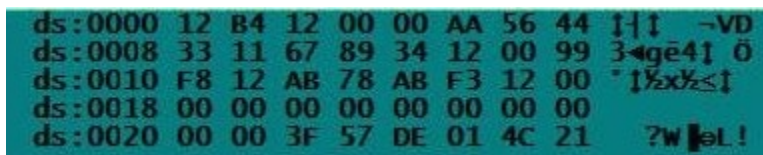
d DQ ?

e DB ?

f DW ?

g DT ?

and the data segment (after the program has been executed) from the next image, indicate the final variable value:



The screenshot shows a debugger window with the following data segment (ds) contents:

| Address | Value                             |
|---------|-----------------------------------|
| ds:0000 | 12 B4 12 00 00 AA 56 44 11 11 -VD |
| ds:0008 | 33 11 67 89 34 12 00 99 34 41 0   |
| ds:0010 | F8 12 AB 78 AB F3 12 00 12 00 1   |
| ds:0018 | 00 00 00 00 00 00 00 00           |
| ds:0020 | 00 00 3F 57 DE 01 4C 21 ?w 1eL!   |

A. 12F8h

B. AB12h

C. F899h

D. 9900j

22. For the next code sequence:

```
XOR CX,CX
XOR AX, AX
```

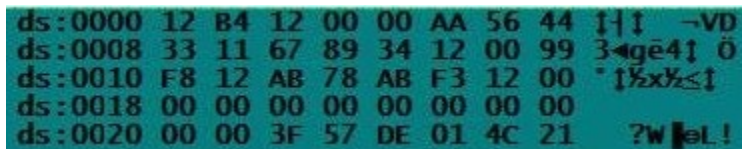
repeat:

```
INC AX
loop repeat
```

indicate which is the AX register value after executing it:

- A. 1
- B. 0
- C. 65535 as a decimal value
- D. The AX value can't be determined because the sequence has an infinite loop
- E. FFFFh

23. Based on the data segment image, indicate which is the DOUBLE value (DD) stored at the DS:[000C] address:



The screenshot shows a memory dump with the following data:

| Address | Hex Value               | ASCII    |
|---------|-------------------------|----------|
| ds:0000 | 12 B4 12 00 00 AA 56 44 | 111 -VD  |
| ds:0008 | 33 11 67 89 34 12 00 99 | 34ge41 0 |
| ds:0010 | F8 12 AB 78 AB F3 12 00 | 11x11    |
| ds:0018 | 00 00 00 00 00 00 00 00 |          |
| ds:0020 | 00 00 3F 57 DE 01 4C 21 | ?wle!    |

- A. 99001234h
- B. 34120099h
- C. 1234h
- D. none of these values
- E. 3412h

24. Which of the following statements it is NOT true regarding the characteristics of a macro-virus:

- A. infect other documents on the host computer
- B. none of these answers
- C. may contain binary executable code as data
- D. is written in MS Office applications suite language - VBA = Visual Basic for Applications
- E. can launch executable viruses
- F. is usually transmitted by e mail

25. X21 virus for Free BSD UNIX operating system, is written in C/C++ and it is a companion virus because:

- a. UNIX operating system is running on a large variety of platforms (microcontrollers AMD / Intel or RISC)
  - b. the ASM compiled source code is faster than the C compiled source code
  - c. in this form it is portable
  - d. it is easier with C/C++ syntax to overwrite the infected file source;
- A. a + c + d                      D. a + b + c + d
- B. b + c                          E. a + c
- C. none of the arguments is correct

**26. The virus structure contains as mandatory routines:**

- a. search**
- b. processing**
- c. infection**
- d. anti-detection**
- e. stealth**

**Select the correct combination:**

- A. a + b + c**
- B. a + c + e**
- C. a + b + c + e**
- D. b + c**
- E. a + c**

**27. To address a memory area using real addressing mode (not in safe mode) there are required two 16-bit data: segment address and offset inside the segment.**

**The physical address ( 20-bit in real addressing mode) is generated automatically by a hardware/software component inside the processor.**

**Knowing that the data segment register, DS has a value equal with 4F1Dh, indicate the results generated by the next sequence:**

**.model small**

**.data**

**x db 8**

**.code**

**start:**

**mov AX, @data**  
**mov DS, AX**

**xor ax, ax**  
**mov ax, 5**  
**inc ax**  
**inc ax**  
**mov BX, 0123h**  
**mov word ptr DS:[BX], AX**

**mov CX, 7**  
**inc CX**  
**mov AX, DS**  
**inc AX**  
**mov DS, AX**  
**mov BX, 0113h**  
**mov word ptr DS:[BX], CX**  
**mov AX, 4C00h**

int 21h  
end start

A. None of these answers is correct

B. At the 4F1Dh:0123h address (seg:off) it is stored 8, and at 4F1Eh:0113h it is stored 7

C. At the 4F1Dh:0123h address (seg:off) it is stored 5, and at 4F1Eh:0113h it is stored 8

D. At the 4F1Dh:0123h address (seg:off) it is stored 7, and at 4F1Eh:0113h it is stored 8

E. At the 4F1Dh:0123h address (seg:off) it is stored 8, and at 4F1Eh:0113h it is stored 8

F. At the 4F1Dh:0123h address (seg:off) it is stored 5, and at 4F1Eh:0113h it is stored 7

28. Considering the next code sequence:

.code

mov AX, @data

mov DS, AX

mov BX, 10

label1: ;start label

xor AX, AX

inc AX

add AX, BX

mov AX, 4c00h

int 21h

end label1

indicate the value of the AX register before executing mov AX, 4c00h

A. 000Bh

B. 0011h

C. you can't determine the value because the result depends on the initial value of BX

D. 0001h

E. 0000h

29. Which statement is correct regarding the next code sequence:

mov AX, @data

mov DS, AX

A. Initialize the code segment register with an address determined at run-time

B. Initialize the code segment register with a constant value, randomly generated by the processor

C. The sequence has assembler errors because @data is not a valid statement

D. The sequence has assembler errors because the 2 operands of the mov AX,@data instruction have different size

E. Initialize the data segment register with the data segment address of the process

F. Initialize the data segment register with a constant value, randomly generated by the processor